WNUT

Security Audit Report

Wrapped NUT - Base Mainnet

PASS	**0 Critical	0 High	0 Medium	0 Low** 4 informational hardening items

Contract	WNUT / Wrapped NUT
Network	Base Mainnet (chain ID 8453)
Address	0xBA5ED46e31F80dafF307cbf60bd422a6f62E3dd3
Underlying	NUT - 0xb8DE15Fb529d98C93C749De63c749D48D25A30DF
Compiler	Solidity 0.8.36
Dependency	OpenZeppelin Contracts v5.7.0
Deployment	CREATE2 via Arachnid deterministic deployment proxy
Report date	2026-08-22
Report ID	WNUT-SEC-2026-08-22

Overall assessment

No exploitable vulnerability was identified in the scoped WNUT wrapper. The contract is a minimal, non-upgradeable, non-administered 1:1 ERC-20 wrapper whose operational logic is inherited from OpenZeppelin ERC20Wrapper v5.7.0. The remaining items are documentation, CI/provenance, and user-error hardening recommendations.

Important: no audit can establish that software is literally “unhackable.” This report records the evidence reviewed, assumptions made, and vulnerabilities identified within the stated scope.

1. Executive Summary

WNUT is intentionally narrow. Its only custom Solidity logic is a constructor that sets the ERC-20 name/symbol and fixes the immutable underlying token. Wrapping and unwrapping are delegated to OpenZeppelin's ERC20Wrapper implementation. There are no privileged roles, upgrade hooks, rescue functions, arbitrary calls, or protocol-specific accounting extensions.

Security conclusion: within the reviewed scope and stated assumptions, the wrapper presents a minimal attack surface and preserves the expected 1:1 raw-unit backing invariant. No Critical, High, Medium, or Low severity vulnerability was identified.

Severity Summary

Critical	High	Medium	Low	Informational
0	0	0	0	4

Key security properties

- Immutable underlying token: the NUT address is stored immutably by ERC20Wrapper.
- No owner/admin: no privileged account can mint, seize reserves, pause, upgrade, or redirect the wrapper.
- Backed minting: NUT is transferred into the wrapper before wNUT is minted.
- Burn-before-release: wNUT is burned before NUT is transferred out.
- No external mint path: the inherited internal _mint function is not exposed through custom privileged logic.
- No proxy/upgradeability: deployed runtime behavior cannot be changed through an implementation slot or admin.
- Standard underlying: NUT is a verified fixed-supply, non-tax ERC-20, which avoids the fee-on-transfer/rebase hazards called out by OpenZeppelin for wrappers.

Qualified conclusion

> **Recommended public security statement**
> wNUT is a non-upgradeable, non-administered 1:1 ERC-20 wrapper over immutable NUT, implemented with OpenZeppelin ERC20Wrapper v5.7.0. It exposes no privileged mint, withdrawal, pause, upgrade, rescue, or ownership path.

2. Scope and Reviewed Artifacts

Canonical source	BASEDNUT/sol-contracts - src/WNUT.sol
Deployment record	deployments/base/WNUT.json
Tests	test/wnut/WNUT.t.sol
Build profile	Foundry profile [profile.wnut]
OZ dependency	openzeppelin-contracts commit cab19933... (Release v5.7.0)

Deployment tx	0x8fda16be305849eecd7cd4fa91b765e6d31672ae17d25b5ae1022809f9dc5403
CREATE2 factory	0x4e59b44847b379578588920cA78FbF26c0B4956C
CREATE2 salt	0x135b208ac2480b0d876944416de16e7a3b9a1f65bf4ce3648080ff0306f03ee4
Init-code hash	0x0333a81c53a016584226e27ef7979f8827093744b1c19cb8866206485151ef54

Canonical WNUT source

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.36;
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC20Wrapper} from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Wrapper.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
contract WNUT is ERC20Wrapper {
    constructor(address nut_) ERC20("Wrapped NUT", "wNUT") ERC20Wrapper(IERC20(nut_)) {}
}
```

Out of scope

- Economic performance, market liquidity, price behavior, MEV, or exchange integration risk.
- Security of external wallets, frontends, RPC providers, bridges, DEXes, or third-party custodians.
- Base L2 consensus/sequencer security and Ethereum settlement security.
- Future contracts that may integrate wNUT.
- Formal mathematical verification with a theorem prover.

3. Methodology

The review combined manual source inspection, dependency review, compiler-version review, invariant reasoning, deployment provenance analysis, and inspection of the project's Foundry tests and CI configuration.

Technique	What was examined
Manual source review	Constructor behavior, inherited callable surface, state mutability, access-control surface, external calls, mint/burn paths, and failure ordering.
Invariant analysis	Reasoned about totalSupply versus underlying NUT balance across deposits, withdrawals, direct transfers, and failed calls.
Dependency review	Reviewed OpenZeppelin ERC20Wrapper v5.7.0 and SafeERC20 behavior at the pinned dependency commit.
Underlying-token compatibility	Reviewed public verified NUT source and metadata: fixed supply, standard ERC-20 transfer semantics, 18 decimals, no transfer tax or rebasing.
Compiler review	Confirmed deployed build targets Solidity 0.8.36 with

Technique	What was examined
	optimizer disabled, viaIR disabled, Cancun EVM, IPFS metadata.
Deployment provenance	Reviewed CREATE2 factory/salt/init-code-hash records and the reproducible init-code hash supplied from the deployment build.
Test review	Reviewed unit and fuzz tests covering metadata, wrap/unwrap, backing, no-admin surface, and direct-transfer behavior.

Review limitations

The existing Foundry test suite and public artifacts were inspected but were not re-executed by this reviewer in the document-generation environment. No Slither, Echidna, Medusa, Halmos, Certora, or theorem-prover run was performed here. The project owner separately reproduced the canonical init-code hash with Foundry/Solc 0.8.36 before this report. Explorer verification of the newly deployed address was reported by the project owner; the review relied primarily on the canonical repository source and deployment provenance record.

4. Architecture and Security Model

4.1 Functional model

```
WRAP
User approves NUT -> WNUT.depositFor(recipient, amount)
                |
                +-- SafeERC20.safeTransferFrom(NUT, caller, WNUT, amount)
                +-- _mint(recipient, amount)

UNWRAP
Holder calls WNUT.withdrawTo(recipient, amount)
                |
                +-- _burn(caller, amount)
                +-- SafeERC20.safeTransfer(NUT, recipient, amount)
```

4.2 Backing invariant

Target invariant: `NUT.balanceOf(WNUT) >= WNUT.totalSupply()`

- Initial state: wrapper supply starts at zero.
- Deposit of x raw units: underlying balance increases by x before wrapper supply increases by x.
- Withdrawal of x raw units: wrapper supply decreases by x before underlying balance decreases by x.
- Failed underlying transfer: SafeERC20 reverts the whole transaction; mint/burn state cannot persist independently.
- Direct NUT transfer to WNUT: backing increases without minting, producing surplus collateral rather than undercollateralization.

Result: under the verified NUT token semantics, the wrapper has no path that creates wNUT without first receiving the corresponding NUT. The invariant can become strictly overcollateralized through accidental direct transfers, but not undercollateralized through WNUT logic.

4.3 Trust assumptions

Assumption	Evidence reviewed	Effect if violated
NUT is standard ERC-20	Verified FixedToken source; no tax/rebase/mint path.	Fee-on-transfer or rebasing behavior could break 1:1 accounting.
Underlying address is correct	Immutable constructor arg and deployment manifest.	Wrong underlying would make the wrapper semantically invalid.
Base/EVM executes Solidity semantics	Platform assumption.	Chain-level compromise is outside contract scope.

5. Adversarial Analysis

Attack vector	Analysis	Assessment
Unauthorized mint	No external/custom mint function. depositFor transfers NUT first, then mints.	Not identified
Reserve theft	No owner, rescue, sweep, arbitrary call, or privileged withdrawal path.	Not identified
Upgrade takeover	No proxy or implementation slot; runtime is non-upgradeable.	Not identified
Reentrancy	External calls are only to immutable NUT. NUT is a standard ERC-20 with no callback/hook behavior. Ordering is transfer-before-mint and burn-before-transfer.	Not exploitable under NUT assumptions
Allowance theft	depositFor always pulls from msg.sender; there is no arbitrary from parameter.	Not identified
Fee-on-transfer undercollateralization	NUT is documented and verified as a normal, no-tax ERC-20.	Not applicable
Rebase-induced insolvency	NUT is fixed-supply and non-rebasing.	Not applicable
CREATE2 factory control	Factory authority ends at deployment; constructor does not assign ownership/admin to msg.sender.	No continuing authority
Selfdestruct / code replacement	WNUT exposes no selfdestruct or upgrade mechanism.	Not identified
Signature / permit abuse	WNUT does not add custom permit/signature logic.	Not applicable

5.1 Reentrancy note

ERC20Wrapper does not add a reentrancy guard. For a generic wrapper around an adversarial callback-capable token, reentrancy deserves deeper modeling. Here the underlying address is immutable and points to NUT, whose verified implementation is a conventional ERC-20 without token-transfer callbacks. Accordingly, a practical reentrancy entry point through the underlying token was not identified.

5.2 CREATE2 provenance

The deterministic deployment mechanism affects where the contract is created, not its post-deployment privilege model. The Arachnid deterministic deployment proxy accepts a 32-byte salt followed by init code and deploys with CREATE2. The repository records the factory, salt, init-code hash, compiler configuration, constructor underlying address, and resulting WNUT address.

```
factory:
0x4e59b44847b379578588920cA78FbF26c0B4956C

salt:
0x135b208ac2480b0d876944416de16e7a3b9a1f65bf4ce3648080ff0306f03ee4

initCodeHash:
0x0333a81c53a016584226e27ef7979f8827093744b1c19cb8866206485151ef54

result:
0xBA5ED46e31F80dafF307cbf60bd422a6f62E3dd3
```

6. Compiler and Dependency Review

6.1 Solidity 0.8.36

The dedicated WNUT Foundry profile pins Solc 0.8.36, disables the optimizer and viaIR, selects Cancun EVM, and uses IPFS bytecode metadata. Solidity 0.8.36 fixed two medium-severity compiler issues affecting earlier versions: a mutual-recursion spilling defect in the IR pipeline and an inheritance-order annotation mutation associated with custom storage layout warnings. WNUT also uses viaIR=false and does not use custom storage layout, further reducing exposure to those classes.

6.2 OpenZeppelin Contracts v5.7.0

The repository pins OpenZeppelin Contracts to commit cab19933c33c2ad1d4c7a84864a3601dddfd16f3, the v5.7.0 release commit. WNUT inherits ERC20Wrapper directly. The reviewed implementation stores the underlying token immutably, uses SafeERC20 for transfers, rejects wrapping to the wrapper itself, burns before withdrawal transfers, and exposes no privileged recovery function unless a derived contract chooses to expose the internal _recover method. WNUT does not expose it.

6.3 Underlying NUT compatibility

The public verified NUT contract is a FixedToken implementation that mints the full 1 NUT supply at construction, has 18 decimals, and exposes standard ERC-20 transfer/allowance functions without a subsequent mint function. Project documentation also identifies NUT as a normal, no-tax ERC-20. These properties are compatible with ERC20Wrapper's 1:1 raw-unit accounting.

7. Test Suite Review

The repository's WNUT test suite covers the most important wrapper invariants and privilege assumptions:

- Metadata, decimals, and exact underlying-token address.
- 1:1 deposit minting.
- Permissionless wrapping for another recipient.
- Zero-value deposit behavior.
- 1:1 withdrawal and over-withdraw revert behavior.
- Supply-versus-backing invariant after mixed deposits/withdrawals.
- Fuzzed wrap/unwrap amounts.
- Accidental direct NUT transfer producing stranded surplus rather than minted wNUT.
- Absence of owner(), transferOwnership(), and public recovery functions.

Test assessment

The tests are well aligned with the actual security model. The main gap is not contract behavior coverage; it is deployment-conformance automation. The exact deployed profile and init-code hash should be enforced in CI so future repository changes cannot silently diverge from the canonical deployment.

8. Findings

No Critical, High, Medium, or Low severity security issue was identified. The following informational items improve operational safety, auditability, and provenance.

I-01 Direct NUT transfers are permanently stranded INFORMATIONAL

Description	Users can transfer NUT directly to the WNUT contract without calling depositFor(). Such a transfer increases the wrapper's NUT balance but does not mint wNUT. WNUT intentionally exposes no public recovery function, so the surplus cannot be reclaimed through WNUT.
Security impact	No attacker profit or insolvency was identified. The wrapper becomes overcollateralized. The risk is user loss through incorrect interaction.
Recommendation	Prominently warn users and integrations not to transfer NUT directly to WNUT. Frontends should implement wrapping through approve + depositFor(). Keep the no-rescue design if minimizing privileged attack surface is the priority.
Status	Accepted design / document prominently

I-02 Default CI does not enforce the deployed WNUT build profile

INFORMATIONAL

Description	The repository has a dedicated [profile.wnut] matching the deployed compiler settings, while the default profile uses different optimizer/viaIR/EVM settings. The reviewed CI workflow runs plain forge build and forge test, which use the default profile.
Security impact	A future source/dependency change could pass CI while the canonical deployment reproduction path is broken. This is a provenance and release-engineering weakness, not a runtime vulnerability in the deployed contract.
Recommendation	Add CI steps using FOUNDRY_PROFILE=wnut and assert the exact init-code hash 0x0333...ef54. Treat any mismatch as a release failure.
Status	Recommended

I-03 Documentation contains stale source-location details INFORMATIONAL

Description	The root README still points WNUT to src/wnut/, while the canonical source is now src/WNUT.sol. The WNUT README also describes the wrapper as '18 lines' although the canonical source is shorter after comment removal.
Security impact	No on-chain security impact. Stale documentation can create ambiguity for reviewers and integrators about which source is canonical.
Recommendation	Update documentation to point to src/WNUT.sol and describe the design as minimal/bare rather than relying on a line count.
Status	Recommended

I-04 No automated live-deployment conformance assertion INFORMATIONAL

Description	The deployment manifest records the address, constructor underlying, CREATE2 salt, init-code hash, and compiler settings, but the reviewed CI workflow does not query Base and compare the live contract's observable configuration or runtime bytecode against expected values.

Security impact	No current vulnerability identified. The gap reduces ongoing assurance that documentation, source, and deployed state remain synchronized.
Recommendation	Add a read-only Base fork or RPC check that asserts code exists at the canonical address, underlying() equals NUT, name/symbol/decimals are expected, and the reproducible creation/runtime artifacts match the canonical deployment record.
Status	Recommended

9. Hardening Recommendations

P0 - Preserve deployed runtime: Do not add owner/admin/rescue/upgrade logic to the deployed canonical wrapper. Any such feature would materially increase attack surface and would require a new address.

P1 - Reproducible-build CI: Run the exact wnut Foundry profile in CI and assert the canonical init-code hash.

P1 - Live conformance test: Use a Base fork/RPC check for code presence, underlying(), name(), symbol(), decimals(), and supply/backing sanity.

P1 - Security documentation: Add SECURITY.md with a reporting contact and clearly document that direct NUT transfers do not mint wNUT.

P2 - Signed release provenance: Create a signed/tagged canonical WNUT release and record the commit, dependency commit, deployment tx, salt, and init-code hash.

P2 - Additional tooling: Optionally run Slither plus invariant fuzzing/formal tools against the minimal wrapper and archive the outputs. These are defense-in-depth; the primary attack surface is already small.

10. Final Verdict

PASS - no exploitable vulnerability identified in scope

WNUT's security strength comes from removing complexity rather than adding defenses: immutable underlying asset, no privileged control, no upgradeability, no custom accounting, and OpenZeppelin-managed transfer/mint/burn ordering. Under the verified standard NUT token semantics, the 1:1 wrapper model is sound. The remaining recommendations are operational and provenance hardening, not reasons to redeploy.

Security properties suitable for publication

- Non-upgradeable.
- No owner or administrator.
- No privileged mint.
- No privileged withdrawal or reserve sweep.
- Immutable underlying NUT address.
- 1:1 raw-unit wrapping and unwrapping.
- OpenZeppelin ERC20Wrapper v5.7.0.
- Deterministic CREATE2 deployment with recorded provenance.

Do not publish the term “unhackable.” A technically defensible formulation is: “minimal immutable attack surface; zero privileged control; 1:1 backing by construction under standard NUT token semantics.”

11. References

R1	Canonical WNUT source [source]
R2	WNUT deployment manifest [source]
R3	WNUT Foundry tests [source]
R4	OpenZeppelin ERC20Wrapper v5.7.0 [source]
R5	OpenZeppelin Contracts 5.7 release [source]
R6	Solidity 0.8.36 release announcement [source]
R7	NUT verified BaseScan source [source]
R8	Arachnid deterministic deployment proxy [source]
R9	WNUT BaseScan contract page [source]
R10	WNUT deployment transaction [source]

12. Disclaimer

This document is an independent AI-assisted security review of the artifacts and evidence identified in the Scope section. It is not a warranty, insurance policy, certification of impossibility of exploitation, or substitute for an engagement by a professional audit firm. Smart-contract security also depends on external assumptions, integration correctness, chain behavior, key management, and future ecosystem changes. No representation is made that all possible vulnerabilities have been discovered.